

SystemVerilog Cheat Sheet

1 Data Types

Logic Types

```
logic a;           // Replaces reg/wire for most use cases
bit [3:0] b;        // 4-bit unsigned
int c;             // 32-bit signed
integer d;          // 32-bit signed (legacy)
real r;            // Floating point
```

Composite Types

```
typedef enum {IDLE, BUSY, DONE} state_t;
typedef struct {bit [7:0] addr; bit [7:0] data;} pkt_t;
typedef union {bit [15:0] word; bit [7:0] byte[2];} data_u;
```

2 Operators

Category	Examples
Arithmetic	+ - * / %
Logical	`&&
Bitwise	`&
Reduction	`&~
Shift	<< >> <<< >>>
Relational	< > <= >= == !=

Conditional `a ? b : c`

3 Procedural Blocks

```
initial begin
    // Runs once
end

always @(posedge clk or negedge rst) begin
    if (!rst) q <= 0;
    else      q <= d;
end

always_comb    // For combinational logic
always_ff      // For sequential logic
always_latch    // For latches
```

4 Tasks & Functions

```
task automatic drive(input bit val);
    sig = val;
endtask

function automatic int add(input int a, b);
    return a + b;
endfunction
```

- `task` → allows time-consuming operations (@, #)
 - `function` → executes in zero simulation time
-

5 Classes & OOP

```
class packet;  
    rand bit [7:0] addr, data;  
    function void display();  
        $display("Addr=%0d Data=%0d", addr, data);  
    endfunction  
endclass
```

```
packet p = new();    // Create object  
p.display();         // Call method
```

Inheritance & Polymorphism

```
class base; virtual function void show(); $display("Base");  
endfunction endclass  
class child extends base; function void show(); $display("Child");  
endfunction endclass  
  
base b = new(); child c = new(); b = c; b.show();    // Outputs "Child"
```

6 Randomization & Constraints

```
class packet;  
    rand bit [7:0] addr;  
    rand bit [3:0] length;  
    constraint c1 { addr inside {[0:15]}; length < 8; }  
endclass
```

```
packet p = new();  
if (!p.randomize()) $display("Randomization failed");
```

7 Interprocess Communication (IPC)

Mailbox

```
mailbox mb = new();  
mb.put(data);    // Producer  
mb.get(data);    // Consumer
```

Semaphore

```
semaphore sem = new(1);  
sem.get(1);      // Acquire lock  
sem.put(1);      // Release lock
```

8 Clocking Blocks

```
clocking cb @(posedge clk);  
    input data_out;  
    output data_in;  
endclocking
```

✓ Synchronizes signal sampling & driving with DUT clock edges.

9 Assertions (SVA)

Immediate Assertion

```
assert (a == b) else $error("Mismatch");
```

Concurrent Assertion

```
property req_gnt;  
    @(posedge clk) req |-> ##1 gnt;  
endproperty  
assert property (req_gnt);
```



◆ 10 Functional Coverage

```
covergroup cg @(posedge clk);  
    coverpoint mode;  
    coverpoint cmd;  
    cross mode, cmd;  
endgroup
```

```
cg c = new();  
c.sample();
```



11 Interfaces

```
interface bus_if(input logic clk);  
    logic [7:0] addr, data;  
    modport master (output addr, data);  
    modport slave  (input addr, data);  
endinterface
```



12 Program Block

```
program test;  
    initial begin  
        $display("Start Testbench");  
    end  
endprogram
```

Used for testbench control — executes after DUT initialization.



13 System Tasks

Task	Purpose
<code>\$display, \$monitor</code>	Print values
<code>\$time, \$stime</code>	Simulation time
<code>\$random,</code> <code>\$urandom_range()</code>	Random generation
<code>\$finish, \$stop</code>	End simulation
<code>\$dumpvars</code>	Waveform dump

14 Coverage System Functions

Function	Description
<code>\$set_coverage_db_name("run .ucdb")</code>	Set coverage file name
<code>\$get_coverage()</code>	Get coverage percentage
<code>\$write_coverage()</code>	Write coverage report
<code>cg.get_coverage()</code>	Instance coverage
<code>cg.sample()</code>	Manual sampling

15 Common UVM Links to SystemVerilog

SystemVerilog Concept	UVM Equivalent
Class	<code>uvm_component</code> , <code>uvm_sequence_item</code>
Mailbox	TLM FIFO / analysis port
Constraint Randomization	<code>uvm_do_with()</code>
Covergroup	Coverage subscriber
Interface	Virtual Interface
Assertion	Checker module
Clocking block	Synchronizes agent activity

*******THE END*******